Portfolio Project MIS450

CSU Global

Professor Marohn

Jasmin A. Frink

5 July

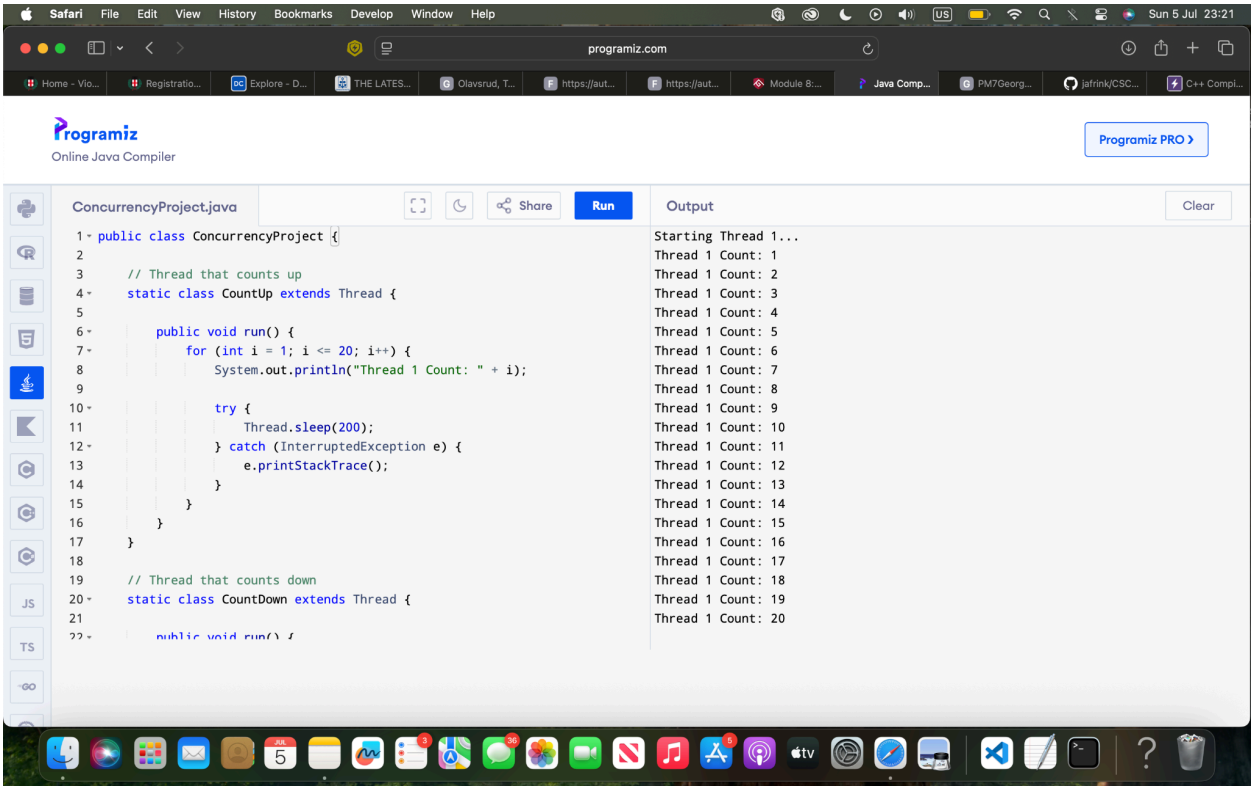
Sourcecode:

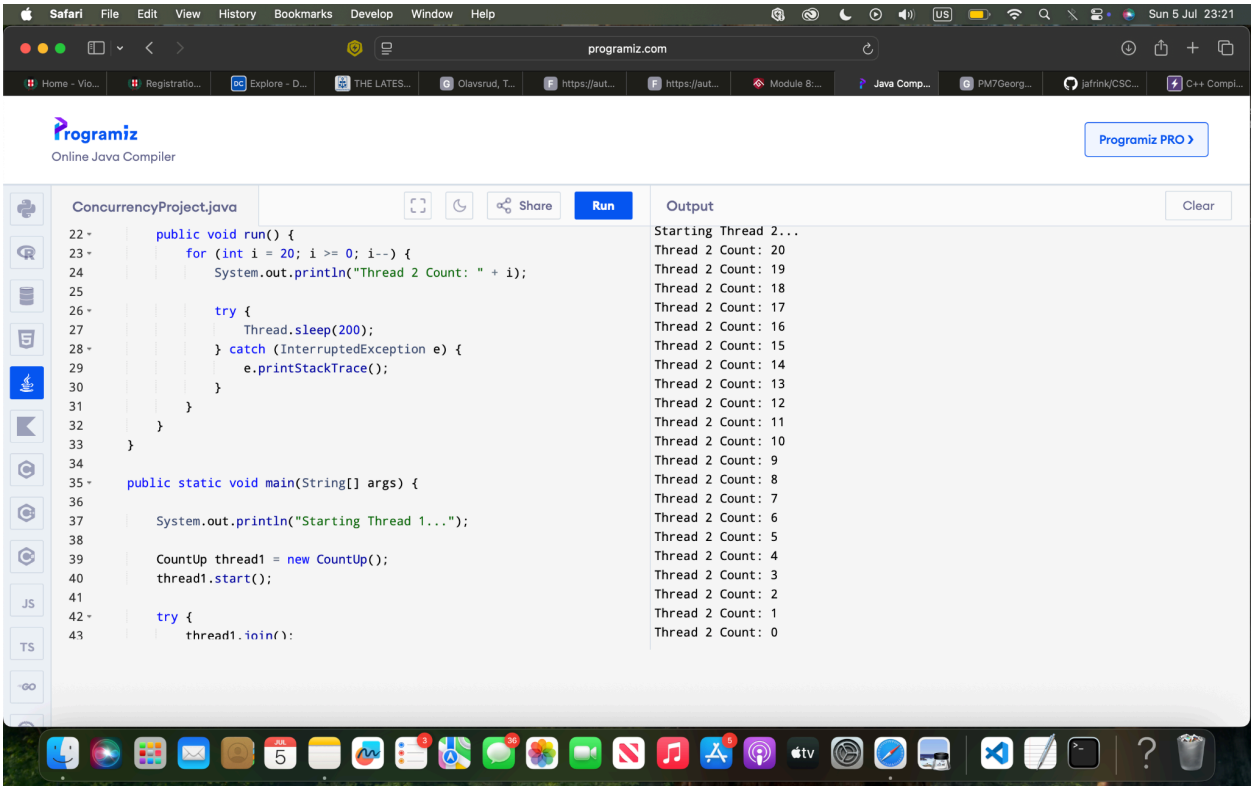
```
public class ConcurrencyProject {

    // Thread that counts up
    static class CountUp extends Thread {

        public void run() {
            for (int i = 1; i <= 20; i++) {
                System.out.println("Thread 1 Count: " + i);

                try {
                    Thread.sleep(200);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        }
    }

    // Thread that counts down
    static class CountDown extends Thread {
```

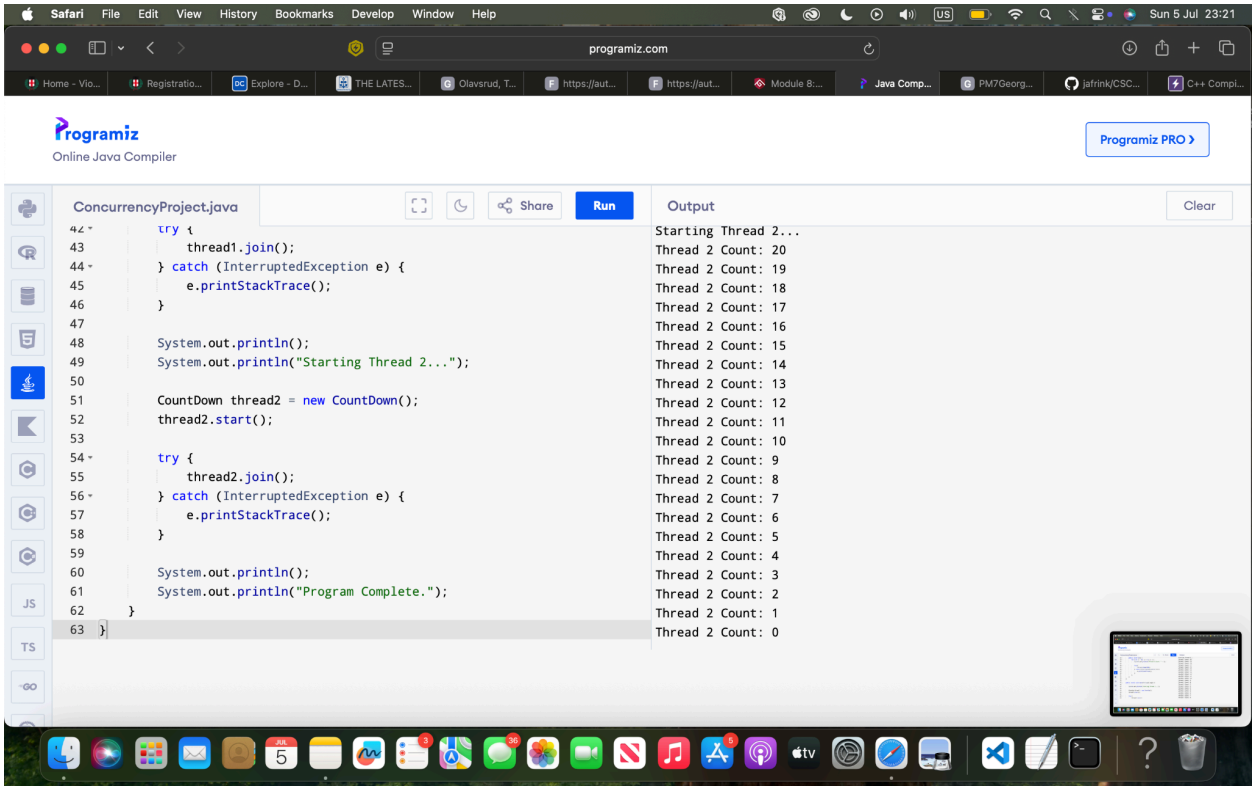
```
    public void run() {
        for (int i = 20; i >= 0; i--) {
            System.out.println("Thread 2 Count: " + i);

            try {
                Thread.sleep(200);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

public static void main(String[] args) {

    System.out.println("Starting Thread 1...");

    CountUp thread1 = new CountUp();
    thread1.start();

    try {
        thread1.join();
```

```
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println();
        System.out.println("Starting Thread 2...");

        CountDown thread2 = new CountDown();
        thread2.start();

        try {
            thread2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println();
        System.out.println("Program Complete.");
    }
}
```

Screenshots:






Programiz
Online Java Compiler
Programiz PRO
ConcurrencyProject.java
Share
Run
Output
Clear
try {
thread1.join();
} catch (InterruptedException e) {
e.printStackTrace();
}
System.out.println();
System.out.println("Starting Thread 2...");
CountDown thread2 = new CountDown();
thread2.start();
try {
thread2.join();
} catch (InterruptedException e) {
e.printStackTrace();
}
System.out.println();
System.out.println("Program Complete.");
}
}
Starting Thread 2...
Thread 2 Count: 20
Thread 2 Count: 19
Thread 2 Count: 18
Thread 2 Count: 17
Thread 2 Count: 16
Thread 2 Count: 15
Thread 2 Count: 14
Thread 2 Count: 13
Thread 2 Count: 12
Thread 2 Count: 11
Thread 2 Count: 10
Thread 2 Count: 9
Thread 2 Count: 8
Thread 2 Count: 7
Thread 2 Count: 6
Thread 2 Count: 5
Thread 2 Count: 4
Thread 2 Count: 3
Thread 2 Count: 2
Thread 2 Count: 1
Thread 2 Count: 0

Analysis

Performance Issues with Concurrency

This program uses two threads to show how concurrency works. The first thread counts from 1 to 20, and once it finishes, the second thread counts from 20 back down to 0. Even though Java is capable of running multiple threads at the same time, I used the join() method so the second thread waits until the first one is finished. This keeps the output organized and makes sure the program follows the assignment requirements.

One thing to keep in mind is that creating and managing threads takes some system resources. Every time a new thread is created, the operating system has to schedule it and keep track of what it is doing. For a small program like this, that extra work is so small that it is almost impossible to notice. However, in a much larger application with dozens or even hundreds of threads running at once, creating too many threads can actually hurt performance. The processor has to spend more time switching between threads instead of doing actual work, which is known as context switching. Overall, concurrency can make programs more efficient when different tasks can run at the same time, but it is important not to create more threads than the application actually needs. In this project, using only two threads keeps the program simple while still demonstrating how concurrency works.

Vulnerabilities Exhibited with Use of Strings

This application only uses strings to print messages to the console, such as "Starting Thread 1" and "Program Complete." Since the program does not ask the user to enter any information, there is very little risk of string-related security problems. The strings are

hard-coded into the program, so they cannot be changed or manipulated while the program is running.

One advantage of Java is that it uses the String class for text instead of character arrays like C++ often does. Java strings are immutable, which means they cannot be changed after they are created. This makes them safer because it prevents accidental changes to data and reduces the chances of certain programming errors. If this program were expanded to accept user input, developers would need to make sure the input is validated before using it. This helps protect against invalid or malicious data that could cause unexpected behavior. Even though that is not an issue in this project, it is still an important security practice whenever a program accepts information from users.

Security of the Data Types Used

The main data type used in this application is the integer (int), which stores the counter values for both threads. Since the counters only go from 0 to 20, there is no realistic chance of integer overflow or other problems related to the size of the data type. The values stay well within the range that an integer can safely store.

Another thing that helps improve the security and reliability of the program is that each thread has its own local counter variable. The threads are not sharing data or trying to change the same variables at the same time. Because of that, there is no risk of race conditions or corrupted data, which are common problems in concurrent programming.

The join() method also helps make the program more reliable by making sure the first thread finishes before the second one starts. This keeps the execution predictable and prevents the threads from interfering with each other. Overall, the data types used in this project are simple, secure, and appropriate for the task, while the program design avoids many of the common issues that can happen when multiple threads are involved.

Comparing the Java and C++ Implementations

Both the Java and C++ versions of this project accomplish the same goal by creating two threads. In each program, the first thread counts from 1 to 20, and after it finishes, the second thread counts from 20 back down to 0. Even though the programs produce the same results, the two languages handle concurrency a little differently because of how they manage memory, threads, and system resources.

One thing I noticed while building both programs is that the overall logic is almost exactly the same. In C++, I used std::thread along with the join() function to make sure the first thread finished before the second one started. In Java, I used the Thread class and the join() method to do the same thing. The syntax is different, but the idea behind it is exactly the same. Personally, I found Java a little easier to read, while C++ gave me more control over how everything worked behind the scenes.

When it comes to performance, C++ usually has a slight advantage. C++ programs are compiled directly into machine code, so they can communicate with the operating system without needing another layer in between. Java programs run inside the Java Virtual Machine (JVM), which adds a little extra overhead. Modern versions of the JVM are very efficient and

use Just-In-Time (JIT) compilation to improve speed, so the difference is usually small. For a simple project like this one, there really is not a noticeable difference. However, in larger applications with many threads or more demanding tasks, C++ often performs a little better because it gives developers more direct control over the computer's resources.

Memory management is another area where the two languages are different. In C++, developers are responsible for managing memory themselves. That flexibility can improve performance, but it also makes it easier to accidentally create problems like memory leaks or invalid memory access if resources are not managed correctly. Java uses automatic garbage collection, which means the JVM automatically cleans up memory that is no longer needed. This makes Java programs easier to maintain and reduces the chances of memory-related bugs. This plays into memory safety as well. C++ allows programmers to work directly with pointers, which gives them more flexibility but also creates more opportunities for programming mistakes that can turn into security vulnerabilities. Java hides pointers from developers, making it much harder to accidentally corrupt memory or access invalid memory locations. This built-in protection makes Java a little safer for many applications.

Furthermore, the two languages also handle strings differently. In the C++ version of this project, I only used simple output messages, so there were no real security concerns. However, C++ does allow programmers to use character arrays, which can create security risks like buffer overflows if they are not handled carefully. Java uses the String class by default, and Java strings are immutable. Since they cannot be changed after they are created, many common string-related programming mistakes are avoided automatically.

Both versions of this project also avoid many common concurrency problems because each thread has its own counter variable. Since the threads are not sharing data, there is no risk of race conditions or corrupted information. Using join()in both languages also guarantees that the second thread does not begin until the first one has completely finished, making the program's behavior consistent every time it runs.

Overall, both languages worked well for this assignment, but they each have different strengths. C++ is usually the better option when performance is the highest priority because it runs directly on the operating system and gives developers more control over system resources. Java, on the other hand, includes several built-in features that make it safer and easier to work with, including automatic garbage collection, immutable strings, and protection from unsafe pointer operations. Because of these features, I think the Java version is less vulnerable to security threats. Even though C++ can be just as secure when written carefully, Java helps prevent many common mistakes before they become problems, making it a strong choice for applications where security and reliability are important.